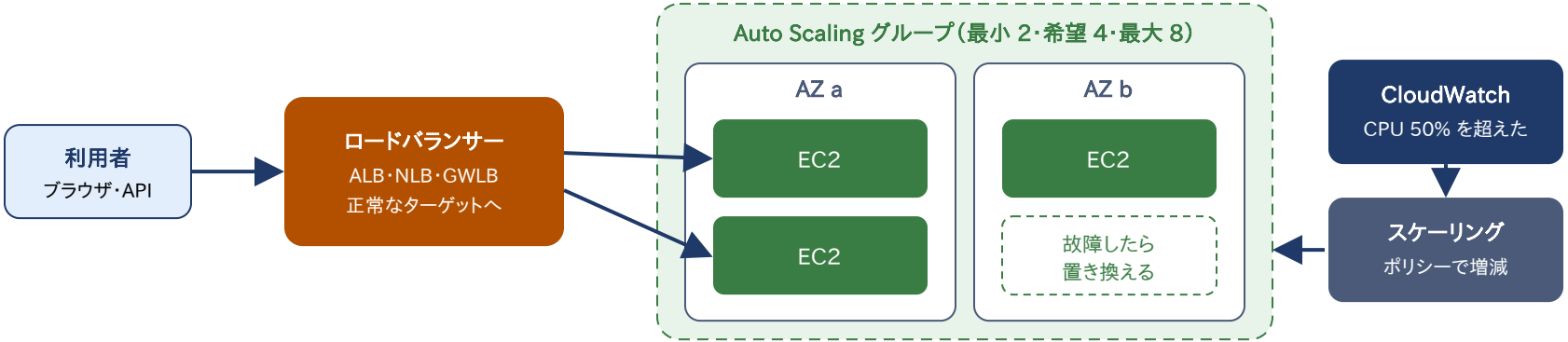
AWS SAA（SAA-C03）　分野2・3 弾力性／高パフォーマンス　#09

ロードバランサーと Auto Scaling

リクエストを複数のサーバーに分け（ELB）、サーバーの台数を負荷に合わせて増減し、壊れたら置き換える（EC2 Auto Scaling）。種類の選び方と設定の決め方

各ページの下に、根拠にした AWS の公式ドキュメント（Elastic Load Balancing・Application／Network／Gateway Load Balancer・Amazon EC2 Auto Scaling）を書いた（青い文字を押すと開く）。「」の中はドキュメントの原文どおり（日本語版は機械翻訳のため、語順が不自然なところもそのまま）。会社名や構成の例は Claude が作った例

全体像 試験ガイド タスク 2.1・2.2・3.2



ELB の役目：複数の AZ のターゲットに分け、ヘルスチェックで正常なものにだけ送る。ロードバランサー自体もトラフィックに合わせて自動でスケールする。

Auto Scaling の役目：負荷に合わせて台数を増減する（弾力性）＋故障したインスタンスを置き換えて希望の数を保つ（耐障害性）。ELB に自動で登録・登録解除する。

出典：Elastic Load Balancing とは? ／ Amazon EC2 Auto Scaling とは（「Amazon EC2 Auto Scaling は、EC2 ヘルスチェックを使用してインスタンスのヘルスと可用性を自動的にモニタリングし、終了したインスタンスまたは障害が発生したインスタンスを置き換えて、希望するキャパシティを維持します。」）

第1章

ロードバランサーの種類

何を見て分けるか（第 7 層か第 4 層か）、固定 IP が要るか、アプライアンスを挟むか。種類の選び方がまず問われる。

4 つの種類を見分ける

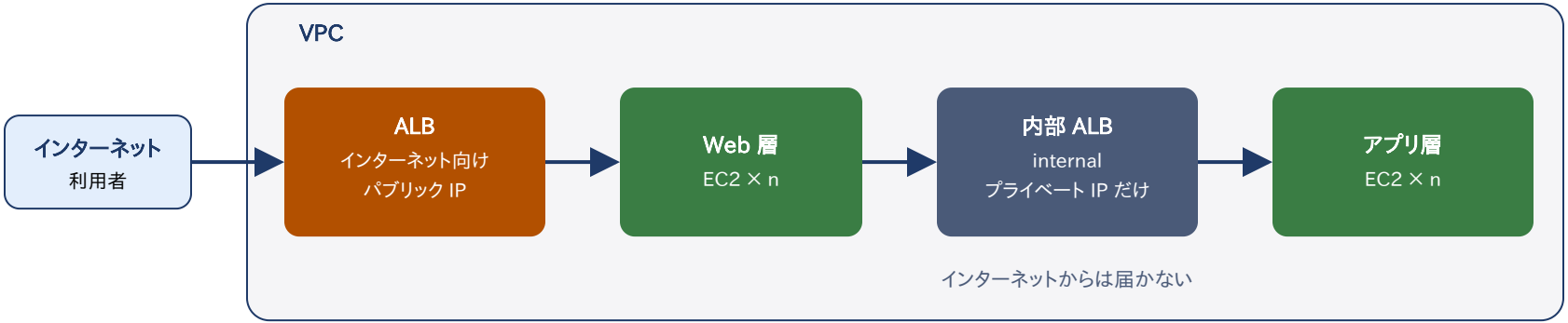
項目	ALB	NLB	GWLB	CLB
層	第 7 層（HTTP・HTTPS）	第 4 層（TCP・UDP・TLS）	第 3 層（GENEVE 6081）	前世代
何で分ける	パス・ホスト・ヘッダー・メソッド・クエリ・送信元 IP	接続（フロー）ごと	通信を丸ごとアプライアンスへ	—
ターゲット	instance・ip・lambda	instance・ip・alb	仮想アプライアンス	—
固定 IP	なし（DNS 名で使う）	AZ ごとに 1 つ・Elastic IP 可	—	—
クロスゾーン	常に有効	デフォルト無効	デフォルト無効	—
本番の言い回し	「URL のパスで」「ホスト名で」「Lambda を後ろに」	「固定 IP」「UDP」「毎秒数百万」「超低レイテンシー」	「サードパーティのファイアウォール」「IDS/IPS を透過的に」	移行元。HTTP→ALB、TCP→NLB

覚え方：中身（URL）を見るなら 7 層の ALB。中身を見ずに速さと固定 IP なら 4 層の NLB。検査装置に通すなら 3 層の GWLB。

ひっかけ：「ALB に Elastic IP を付ける」はできない。固定 IP と ALB の機能を両方ほしいなら、NLB のターゲットに ALB を登録する。

出典：Application Load Balancer とは?（「Application Load Balancer は、開放型システム間相互接続 (OSI) モデルの第 7 層であるアプリケーションレイヤーで機能します。」）／Network Load Balancer とは?（「Network Load Balancer は、開放型システム間相互接続 (OSI) モデルの第 4 層で機能します。」）／Gateway Load Balancer とは?（「ゲートウェイロードバランサーは、開放型システム間相互接続 (OSI) モデルの第 3 層（ネットワークレイヤー）で機能します。」）

スキーム：インターネット向けと内部向け

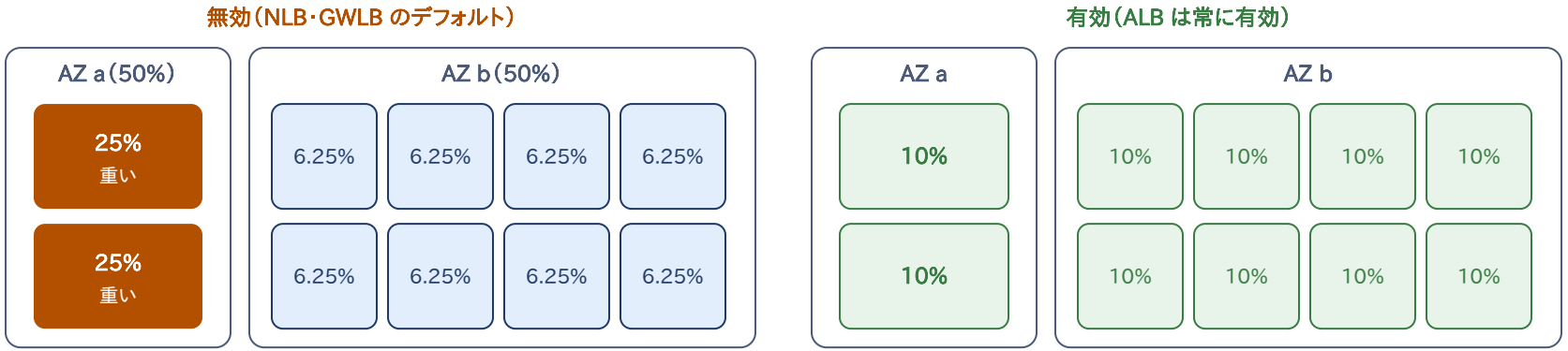


作るときに決める：インターネット向けはノードにパブリック IP があり、DNS 名がパブリックに解決される。内部向けはプライベート IP だけで、VPC の中（や VPN・Direct Connect でつながった先）から使う。

本番の言い回し：「アプリ層は Web 層からだけ受ける」→ 内部向けのロードバランサー。インターネット向けにしてセキュリティグループで絞るより筋がよい。

出典：Elastic Load Balancing の仕組み（「ロードバランサーを作成するとき、ロードバランサーを内部向けにするかインターネット向けにするか選択する必要があります。」「インターネット向けロードバランサーのノードにはパブリック IP アドレスが必要です。」）

クロスゾーン負荷分散：AZ ごとの台数の差



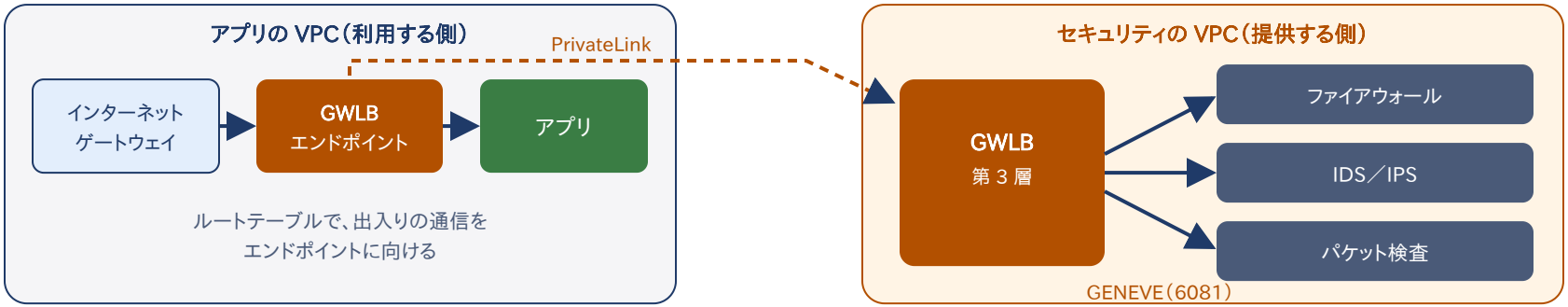
2 つの AZ のノードにトラフィックが半分ずつ来るとした計算（Claude が作った例）。無効だと各ノードは自分の AZ のターゲットにだけ送るので、台数の少ない AZ a の 1 台が 25% を受け持つ。

答え方：「NLB の後ろで一部の AZ のインスタンスだけ負荷が高い」→ クロスゾーン負荷分散を有効にする（または AZ ごとの台数をそろえる）。

AZ の数：ALB は少なくとも 2 つの AZ を有効にする必要がある。1 つの AZ だけのロードバランサーは作れない。

出典：Elastic Load Balancing の仕組み（「Network Load Balancer と Gateway Load Balancer では、クロスゾーン負荷分散はデフォルトで無効になっています。」「Application Load Balancer では、クロスゾーン負荷分散がロードバランサーレベルで常に有効になっています。」「ただし、Application Load Balancer では、少なくとも 2 つ以上のアベイラビリティーゾーンを有効にする必要があります。」）

GWLB：検査装置を透過的に挟む



何をする：ファイアウォール・侵入検知と防止・ディープパケットインスペクションなどの仮想アプライアンスをまとめて置き、台数を増減し、正常なものにだけ通す。通信は検査して元の流れに戻る（透過的）。

ほかとの違い：AWS WAF は ALB・CloudFront などの HTTP のリクエストを調べる。Network Firewall は AWS のマネージドのファイアウォール。「サードパーティのアプライアンス」なら GWLB。

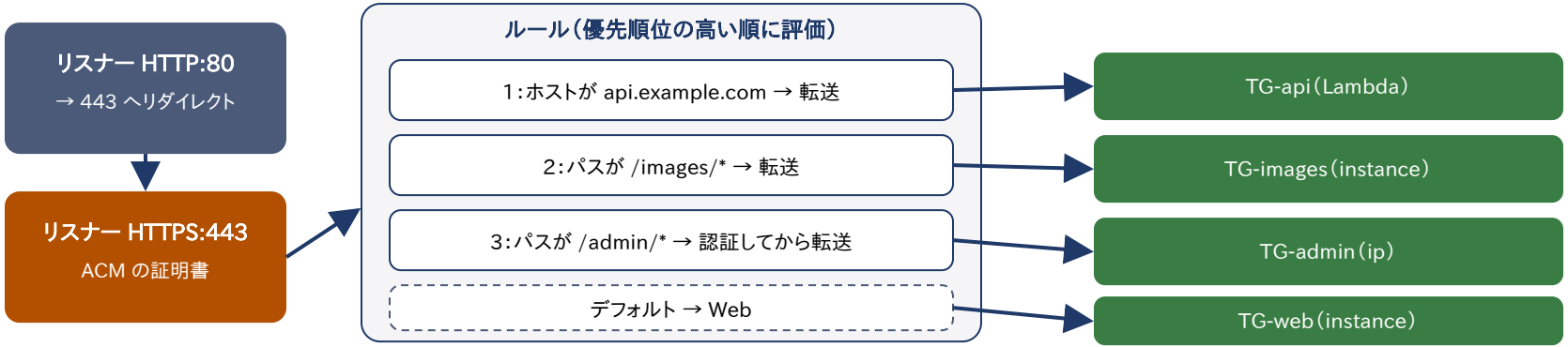
出典：Gateway Load Balancer とは？（「ゲートウェイロードバランサー を使用すると、ファイアウォール、侵入検知および防止システム、ディープパケットインスペクションシステムなどの仮想アプライアンスをデプロイ、スケーリング、管理できます。」「Gateway Load Balancer エンドポイントとの間で送受信されるトラフィックは、ルートテーブルを使用して設定されます。」）

第2章

ALB と NLB の設定

入口（リスナー）で何を見て、どのターゲットグループに送るか。外すとき・入れるとき・同じ人をどう扱うか。

リスナー → ルール → ターゲットグループ



条件（何を見る）	アクション（何をする）
ホスト・パス・HTTP ヘッダー・メソッド・クエリ文字列・送信元 IP	転送（forward）／リダイレクト（http→https など）／固定レスポンス／ユーザー認証（authenticate-cognito・authenticate-oidc）

出典：Application Load Balancer とは?（「リスナールールを構成し、アプリケーショントラフィックのコンテンツに基づいて異なるターゲットグループにリクエストをルーティングできます。」）／リスナールールの条件の種類／リスナールールのアクションタイプ（「1 つの URL から別の URL にリクエストをリダイレクトします。」）

ターゲットグループとターゲットタイプ

タイプ	登録するもの	こんなとき
instance	EC2 インスタンスの ID	ふつうの EC2・Auto Scaling グループ
ip	IP アドレス（VPC の CIDR、ピアリング先、Direct Connect・Site-to-Site VPN でつながったオンプレミス）	Fargate のタスク（awsvpc）、オンプレミスのサーバー、移行の途中
lambda	Lambda 関数 1 つ（ALB だけ）	サーバーなしで HTTP に応える
alb	Application Load Balancer（NLB だけ）	固定 IP（NLB）＋パスの振り分け（ALB）

移行の途中で、同じ ALB からオンプレミスと AWS に分ける（例）



作ったあとは変えられない：ターゲットタイプはターゲットグループを作るときに決め、あとで変更できない。変えるならターゲットグループを作り直す。

1 つのターゲットグループの中身はそろえる：同じアプリの同じ役割のものをまとめ、ヘルスチェックや振り分けの設定もターゲットグループごとに決める。

出典：Application Load Balancer のターゲットグループ（「ターゲットの種類が ip の場合、次のいずれかの CIDR ブロックから IP アドレスを指定できます。」「ターゲットグループを作成した後で、ターゲットタイプを変更することはできません。」）／Network Load Balancers のターゲットグループ

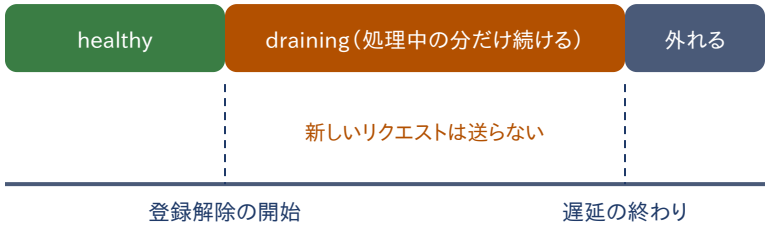
ヘルスチェックと登録解除の遅延

ヘルスチェックの状態

状態	意味
initial	登録中か、最初のヘルスチェック中。1 回合格すると正常になる
healthy	正常。リクエストを送る
unhealthy	応答しないか不合格。送らない
draining	登録解除中。新しいリクエストは送らず、処理中のものが終わるのを待つ

フェイルオープン: 全ターゲットが異常になったら、ALB は状態にかかわらず全ターゲットに送る。設定ミスで全滅してもサービス全体は止めないため。

登録解除の遅延（デフォルト 300 秒）



処理中のリクエストもつながった接続もなくなれば、遅延の終わりを待たずにすぐ外れる。

答え方:「スケールインやデプロイで処理中のリクエストがエラーになる」→ 登録解除の遅延を処理時間より長く。逆に「外れるのが遅くデプロイに時間がかかる」→ 短く。

出典：Application Load Balancer ターゲットグループのヘルスチェック（「ターゲットは、登録後に正常と見なされるためには、1 つのヘルスチェックに合格する必要があります。」「ターゲットグループに異常な登録済みターゲットのみが含まれている場合、そのヘルスステータスにかかわらず、ロードバランサーはそれらすべてのターゲットにリクエストをルーティングします。」）／Application Load Balancer のターゲットグループ属性を編集する（「デフォルトでは、Elastic Load Balancing 登録解除プロセスを完了する前に 300 秒待って、ターゲットへ処理中のリクエストが完了するのを助けることができます。」）

同じ人・空いている所・新入り：ターゲットグループの属性

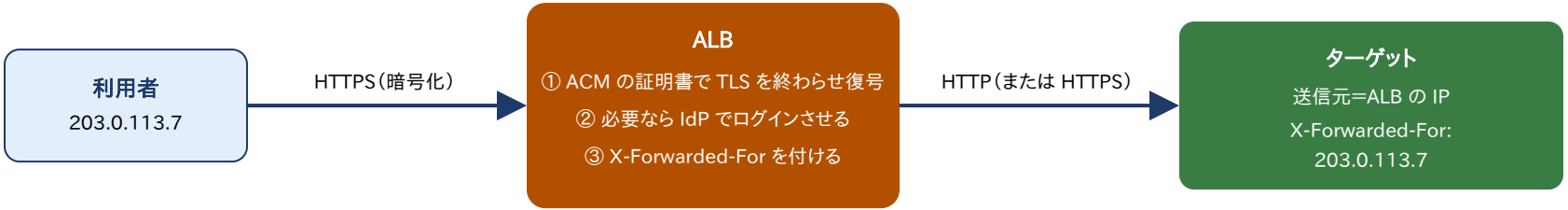
属性	何をする	本番の言い回し
スティッキーセッション	Cookie で同じクライアントを同じターゲットへ。期間ベース（ALB の Cookie）とアプリケーションベース（アプリの Cookie）。クライアントが Cookie に対応している必要がある	「サーバーのメモリにセッション」「コードを変えずにログインを保つ」
ラウンドロビン	順番に均等に送る。デフォルト	—
最小の未処理のリクエスト	処理中のリクエストが最も少ないターゲットへ	「重い処理と軽い処理が混ざる」「処理時間がまちまち」
加重ランダム	ランダムな順で均等に送る	—
スロースタートモード	追加したターゲットへのリクエスト数を直線的に増やす。最小の未処理のリクエスト・加重ランダムとは一緒に使えない	「起動直後はキャッシュが空で遅い」「ウォームアップが要る」

スティッキーの弱点：負荷が偏り、そのターゲットが落ちるとセッションも消える。長い目では、セッションを DynamoDB や ElastiCache に置いてステートレスにする（どのターゲットでも同じ）。

まぎらわしい組：「外すときに待つ」＝登録解除の遅延。「入れたときに少しずつ」＝スロースタート。「同じ人を同じ所へ」＝スティッキー。

出典：Application Load Balancer のターゲットグループ属性を編集する（「スティッキーセッションを使用するには、クライアントが Cookie をサポートする必要があります。」「最も未処理のリクエストのルーティングアルゴリズムは、進行中のリクエストの数が最も少ないターゲットにリクエストをルーティングします。」「ロードバランサーは、スロースタートモードのターゲットに送信できるリクエスト数を直線的に増加させます。」「最小の未処理のリクエストのルーティングアルゴリズムは、スロースタートモードでは使用できません。」）

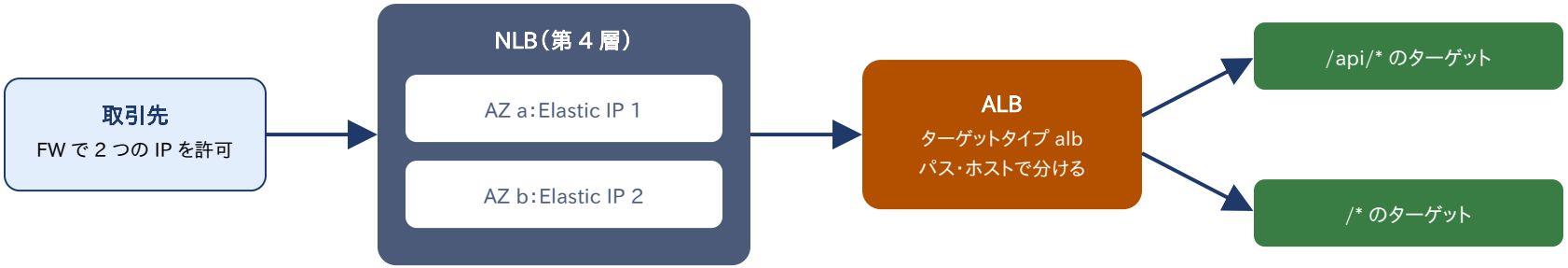
HTTPS の終端・X-Forwarded-For・ユーザー認証



機能	要点	本番の言い回し
HTTPS リスナー	証明書を少なくとも 1 つ。ACM なら無料で自動更新。複数の証明書を SNI で使い分ける	「暗号化の負荷を EC2 から外す」「証明書の更新の手間」
X-Forwarded-For	元のクライアントの IP。ほかに X-Forwarded-Proto・X-Forwarded-Port	「ログの送信元がすべて ALB の IP」
ユーザー認証	authenticate-oidc（OIDC の IdP）・authenticate-cognito（Cognito のユーザープール）	「アプリを変えずにログイン」

出典：Application Load Balancer 用の HTTPS リスナーを作成する（「ロードバランサーはサーバー証明書を使用してフロントエンド接続を終了してから、ターゲットにリクエストを送信する前に、クライアントからのリクエストを復号します。」）／HTTP ヘッダーと Application Load Balancer（「X-Forwarded-For リクエストヘッダーは、HTTP または HTTPS ロードバランサーを使用する場合に、クライアントの IP アドレスを識別するのに役立ちます。」）／Application Load Balancer を使用してユーザーを認証する

NLB の固定 IP と「NLB の後ろに ALB」



NLB の IP：AZ ごとのノードが固定の IP アドレスを持つ。インターネット向けなら、サブネットごとに Elastic IP を 1 つ付けられる。ALB の IP は変わるので、IP を登録してもらう相手には向かない。

世界中から固定 IP：複数のリージョンをまたいで固定の IP で入り口を作るなら Global Accelerator（#19 で学ぶ）。1 つのリージョンなら NLB で足りる。

出典：Network Load Balancer とは?（「インターネット向けのロードバランサーを作成する場合は、必要に応じて 1 つの Elastic IP アドレスをサブネットごとに関連付けることができます。」「1 秒あたり数百万のリクエストを処理できます。」）／Network Load Balancers のターゲットグループ

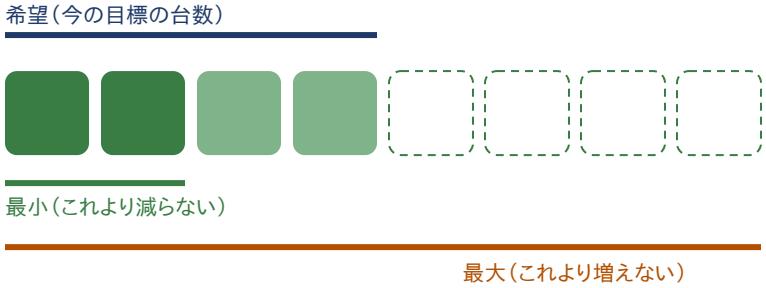
第3章

EC2 Auto Scaling

台数の枠（最小・希望・最大）を決め、何を見て増減するかを選ぶ。起動・終了の途中に処理をはさみ、起動の遅さを補う。

Auto Scaling グループと 2 つのスケーリング

最小・希望・最大（最小 2・希望 4・最大 8 の例）



ポリシーは最小と最大のあいだで「希望」を動かす。ポリシーがなくても、故障したら置き換えて希望の台数を保つ。

	水平（台数）	垂直（大きさ）
言葉	スケールアウト／イン	スケールアップ／ダウン
上限	高い	最大のタイプまで
障害	1 台落ちても続く	1 台なら止まる
変え方	Auto Scaling で自動	止めてタイプ変更が多い
前提	ステートレスに作る	—

AZ のバランス：複数の AZ のサブネットを指定すると、AZ のあいだで台数をそろえる。1 つの AZ が落ちても残りで続けられる。

出典：Amazon EC2 Auto Scaling とは／Auto Scaling グループのスケーリング制限を設定する／「Auto Scaling グループ」

起動テンプレート・ヘルスチェック・猶予期間

設定	要点	本番の言い回し → 答え
起動テンプレート	AMI・インスタンスタイプ・セキュリティグループ・ユーザーデータなど。起動設定と違い、複数のバージョンを持てる。複数のタイプやスポットの組み合わせもこちら	「AMI を新しくしたい」→ 新しいバージョン+インスタンスの更新
EC2 のヘルスチェック	デフォルト。インスタンスが動いているか（ステータスチェック）だけを見る	「アプリが固まっても置き換わらない」→ これだけでは足りない
ELB のヘルスチェック	有効にすると、ロードバランサーが異常と報告したインスタンスも置き換える	「ALB では異常なのに置き換わらない」→ 有効にする
猶予期間	起動直後、異常でも終了しない最小の時間。コンソールのデフォルト 300 秒	「準備中に終了と起動をくり返す」→ 準備時間より長く

くり返しのわな：猶予期間が短いと、起動 → 準備中にヘルスチェック不合格 → 終了 → また起動、がずっと続く。ELB のヘルスチェックを外すのは答えにならない（固まったときに置き換わらなくなる）。

デフォルトのインスタンスウォームアップ：新しいインスタンスのメトリクスが落ち着くまで集計に入れない時間。ターゲット追跡・ステップでの増やしすぎを防ぐ。

出典：Auto Scaling 起動テンプレート（「ただし、起動設定の代わりに起動テンプレートを定義すると、複数のバージョンの起動テンプレートを使用することができます。」）／Auto Scaling グループのヘルスチェックについて／Auto Scaling グループにヘルスチェックの猶予期間を設定する（「コンソールでは、Auto Scaling グループを作成するときのヘルスチェックの猶予期間がデフォルトで 300 秒になっています。」）

スケーリングポリシーを選ぶ

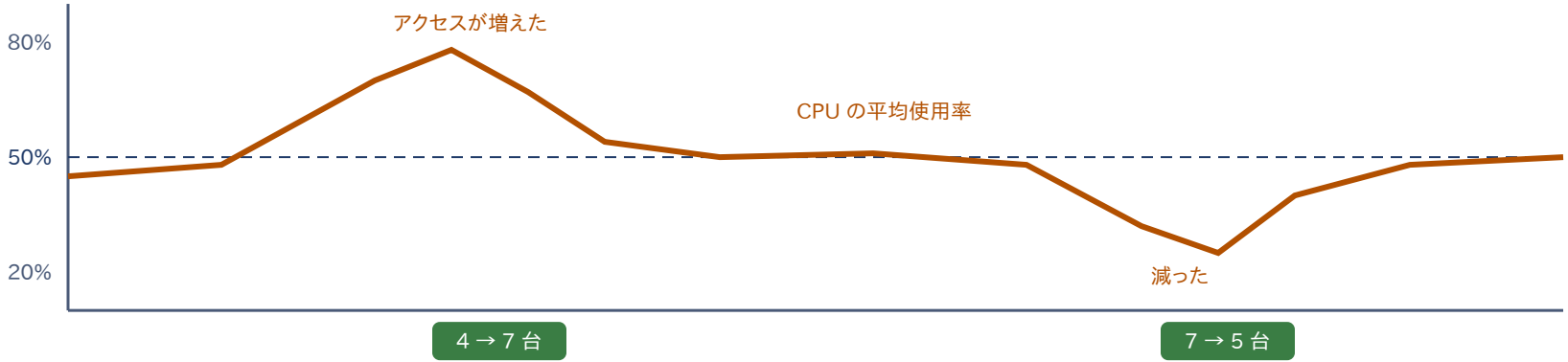
方法	決め方	本番の言い回し
ターゲット追跡	メトリクスと目標値だけ。サーモスタットのように保つ。アラームは自動で作られる	「CPU を 50% くらいに保つ」「最も簡単に」
ステップ	アラームのしきい値をどれだけ超えたかで、増減する数を段階的に	「大きく超えたら一気に増やす」
シンプル	アラーム 1 つで決めた数。1 回動くとクールダウンが終わるまで待つ	AWS は使わないことを勧める
スケジュール	日時（cron とタイムゾーン）で最小・希望・最大を設定	「毎週月曜の朝」「年度末」「セールの前」
予測	過去の負荷から日次・週次のパターンを見つけ、前もって増やす	「毎日くり返す山」「起動が遅く、山の前に準備」

反応か予定か：ターゲット追跡・ステップ・シンプルは「負荷が来てから」の動的スケーリング。スケジュールは人が日時を決める。予測は AWS が過去から学ぶ。

ひっかけ：年に 1 回の決まった日の山は、予測（日次・週次のくり返し）よりスケジュール。「希望を最大にしておく」は常に最大でコストのむだ。

出典：Amazon EC2 Auto Scaling のターゲットトラッキングスケーリングポリシー（「例として、これはサーモスタットがターゲット温度を維持する仕組みと似ています。」）／Amazon EC2 Auto Scaling のステップおよび簡易スケーリングポリシー（「ベストプラクティスとして、シンプルスケーリングポリシーとスケーリングクールダウンを使用しないことをお勧めします。」）／スケジュールされたスケーリング／予測スケーリング（「予測スケーリングは、負荷の履歴データを分析して、トラフィックフローの日ごとまたは週ごとのパターンを検出することで機能します。」）

ターゲット追跡の動き（CPU 50% を目標にした例）



数値は Claude が作った例。台数は目標に近づくよう AWS が計算する。

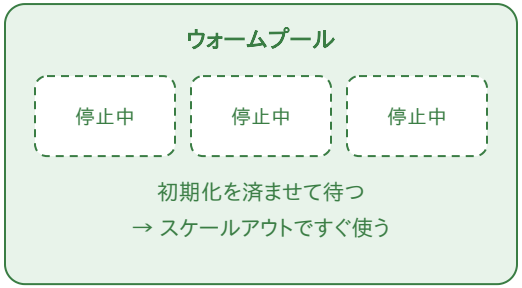
使えるメトリクス：CPU の平均使用率・ネットワークの入出力・ALB のターゲットあたりのリクエスト数など。メトリクスの値は、台数に比例して増減するものを選ぶ。

メッセージの数で：SQS のキューの深さで増減するなら、「インスタンスあたりのバックログ」を自分で計算したメトリクスを目標にする（#06）。

出典：Amazon EC2 Auto Scaling のターゲットトラッキングスケーリングポリシー（「このニーズを満たすには、50% の平均 CPU 使用率をターゲットとする、ターゲット追跡スケーリングポリシーを作成します。」「次に、CPU が 50% を超えると、Auto Scaling グループがスケールアウトして (キャパシティを増やして) 負荷の増加に対応します。」）

ライフサイクルフックとウォームプール





ライフサイクルフック：次の状態に移る前に待たせる。待つ時間のデフォルトは 1 時間。EventBridge で通知し、Lambda を呼んだり、インスタンスに入って作業したりする。「終了前にログを S3 へ」「準備が終わってからトラフィックを受ける」。

ウォームプール：大量のデータを書き込むなど起動が非常に遅いアプリ向け。いつも余分に動かしておく（過剰なプロビジョニング）より安い。起動が速いアプリでは要らない。

出典：Amazon EC2 Auto Scaling のライフサイクルフック（「ライフサイクルフックのデフォルトタイムアウトは 1 時間です (ハートビートタイムアウト)。」「スケールイン イベントが発生すると、ライフサイクルフックによってインスタンスが終了される前に一時停止され、Amazon EventBridge を使用して通知が送信されます。」）／ウォームプールを使用してブート時間が長いアプリケーションのレイテンシーを低減する（「ウォームプールを使用すると、インスタンスが大量のデータをディスクに書き込む必要があるなど、起動時間が非常に長いアプリケーションのレイテンシーを低減できます。」）

どれを終了するか・何で動かすか・どう入れ替えるか

機能	要点	本番の言い回し
終了ポリシー	スケールインでどれを先に終了するか。まず AZ のあいだのバランスを保つ（多い AZ から減らす）。デフォルトは古い設定（古い起動設定・古いテンプレートのバージョン）のものから	「どのインスタンスが終了されるか」
複数のインスタンスタイプと購入オプション	1 つのグループでオンデマンドとスポット、複数のタイプを組み合わせる。キャパシティの再調整で、中断されそうなスポットを前もって置き換える	「最低限はオンデマンド、増える分はスポット」
インスタンスの更新	新しい起動テンプレートのバージョンで、最小正常率と最大正常率に基づき、バッチごとに順に置き換える	「新しい AMI に、止めずに全台入れ替え」

オンデマンドとスポットの組み合わせ（例）：オンデマンドの基本の台数 2・それを超える分はオンデマンド 30%・スポット 70%、タイプは m7i・m6i・m7a から（Claude が作った例）。タイプを増やすほどスポットを確保しやすい。

ひっかけ：「希望を 0 にしてから戻す」「全台を手で終了」はサービスが止まる。「リザーブドをスポットに変換」という機能はない。

出典：Amazon EC2 Auto Scaling の終了ポリシーを設定する（「デフォルトでは、Amazon EC2 Auto Scaling は、古い設定を使用しているインスタンスを最初に終了するように設計された終了ポリシーを使用します。」）／複数のインスタンスタイプと購入オプションを使用する Auto Scaling グループ（「1 つの Auto Scaling グループ内で、オンデマンドインスタンスとスポットインスタンスのフリートを起動してオートスケールできます。」）／Auto Scaling グループでのインスタンスの更新の仕組み（「最小正常率および最大正常率に基づいて、バッチ内のインスタンスを置き換えます。」）

まとめ

種類
7 層で中身を見るなら ALB、4 層で固定 IP・UDP・超大量なら NLB、検査装置を挟むなら GWLB。CLB は前世代

ALB と NLB の設定
ルール（ホスト・パス）→ ターゲットグループ（instance・ip・lambda・alb）。登録解除の遅延・スティッキー・最小の未処理・スロースタート

Auto Scaling の枠
最小・希望・最大。起動テンプレート。ELB のヘルスチェックと猶予期間

増やし方と運用
まずターゲット追跡。日時ならスケジュール、くり返しなら予測。フック・ウォームプール・更新

ひっかけ	正しくは
ALB に Elastic IP	NLB（後ろに ALB）
NLB で AZ の偏り	クロスゾーンを有効に
全ターゲット異常で全部 503	全ターゲットに送る
固まったのに置き換わらない	ELB のヘルスチェック
年 1 回の山に予測スケーリング	スケジュール
ログの送信元が ALB	X-Forwarded-For

次の #10 では、高可用性と災害対策を学ぶ。

出典：このPDFの各ページの資料